

Prompt for Test Strategy document

Context- Act as Experienced E-commerce Specialist TEST Manager

Design a Test Strategy Document for B2B Portal with Hybris and SAP ERP knowledge

[MANDATORY] Ask me 5 valid multiple choice questions

[STRICT] Once after answering first question go to next question

[CRITICAL] Once after answering all the questions generate a test strategy document in excel format

Document Generated

Section	Details
Project Type	B2B Portal with SAP Commerce (Hybris) and SAP ERP
Business Model	Manufacturer to Distributor
Integration Landscape	SAP Commerce Cloud with CPI Integration
Testing Scope	End-to-End Testing including SAP ERP Order Flow
Automation Approach	Manual Testing Only
Priority Functionalities	Quote Management, Approval Workflow & Purchase Orders
Objective	Ensure stable and defect-free B2B commerce transactions integrated with SAP ERP systems.
Test Types	Functional, Integration, Regression, End-to-End, UAT Support
Key Integrations	SAP CPI, SAP ERP, Pricing Engine, Order Management
Test Deliverables	Test Strategy, Test Plan, Test Cases, RTM, Defect Reports, Test Closure Report

Prompt for Test Strategy document

Context- Act as Experienced E-commerce Specialist QA Manager

Design a Test Strategy Document for B2B Portal with Hybris and SAP ERP knowledge [MANDATORY]Ask me 5 valid multiple choice questions

[STRICT] Once after answering first question go to next question

[CRITICAL] Once after answering all the questions generate a test strategy document in .docx format

Document Generated



PROMPT for Automation Framework

Act as an Automation Architect to design the framework for Cypress with Typescript

Instructions

[MANDATORY] Ask me 5 multiple choice questions one by one

[MANDATORY] Once after answering the first question ,proceed with net

[MANDATORY] Generate Framework Structure First After completing all the questions

[STRICT] Proceed Step by Step for framework implementation

[STRICT] Up on my approval proceed with the next step



Using TL-DR; and ELI 15 the above framework is streamlined as below-

tl-dr;

TL;DR:

We'll build a Cypress + TypeScript UI Automation Framework using:

- **Page Object Model**
- **Mochawesome reporting**
- **Azure DevOps pipeline**
- **CI/CD-driven environment configuration**

Framework flow:

- 1. Create project**

- 2. Install Cypress + TypeScript**
- 3. Configure Cypress**
- 4. Add Page Object Model structure**
- 5. Add reusable commands/utilities**
- 6. Add sample UI tests**
- 7. Add Mochawesome reports**
- 8. Add Azure DevOps pipeline**

Step 1 approval needed

Run these commands:

bash

```
mkdir cypress-typescript-ui-frameworkcd cypress-typescript-ui-frameworknpm init -ynpm install cypress typescript --save-devnpx cypress open
```

Then add this to package.json:

json

```
{ "scripts": {  "cy:open": "cypress open",  "cy:run": "cypress run"  }}
```

After this, approve Step 1 and we'll move to Step 2: Cypress Configuration.

approved step 1

Step 2: Cypress Configuration

Create or update cypress.config.ts in the project root.

ts

```
import { defineConfig } from "cypress";export default defineConfig({  e2e: {    baseUrl: process.env.BASE_URL,    specPattern: "cypress/e2e/**/*.cy.ts",    supportFile: "cypress/support/e2e.ts",    setupNodeEvents(on, config) {      return config;    },  },  screenshotsFolder: "cypress/screenshots",  videosFolder: "cypress/videos",  downloadsFolder: "cypress/downloads",  video: true,  screenshotOnRunFailure: true,
```

```
retries: {  runMode: 1,  openMode: 0, },
defaultCommandTimeout: 10000, pageLoadTimeout: 60000,
requestTimeout: 15000, responseTimeout: 30000, viewportWidth:
1440, viewportHeight: 900,});
```

Why this setup?

- **baseUrl: process.env.BASE_URL supports Azure DevOps pipeline variables**
- **specPattern looks for test files under cypress/e2e**
- **supportFile loads reusable commands and global setup**
- **retries.runMode: 1 helps reduce minor CI flakiness**
- **Screenshots and videos are enabled for failure debugging**

Update package.json

Add these scripts:

json

```
{ "scripts": {  "cy:open": "cypress open",  "cy:run": "cypress run",
"test:e2e": "cypress run",  "test:e2e:chrome": "cypress run --
browser chrome"  }}
```

Validate Step 2

Run locally with a temporary BASE_URL.

macOS/Linux

bash

```
BASE_URL=[example.com](https://example.com) npm run cy:run
```

Windows PowerShell

powershell

```
$env:BASE_URL="[example.com](https://example.com)"; npm run
cy:run
```

Expected result: Cypress starts successfully and looks for specs under:

text

cypress/e2e//*.cy.ts**

Approve Step 2 once this is done, and we'll proceed to Step 3: TypeScript Configuration.

ELI 15

Think of Step 2 as telling Cypress: "Where are my tests, what website should I test, and how should Cypress behave?"

Here's the simple version.

What cypress.config.ts does

This file is the main settings file for Cypress.

ts

baseUrl: process.env.BASE_URL

This means:

***Don't hardcode the website URL in the framework.
Get it from the pipeline or terminal instead.***

So locally you can run against:

text

example.com

But in Azure DevOps, the pipeline can provide:

text

qa.yourapp.com

That keeps the framework flexible.

Important settings

ts

specPattern: "cypress/e2e//*.cy.ts"**

This tells Cypress:

Look for test files inside cypress/e2e, and only run files ending with .cy.ts.

Example:

text

cypress/e2e/login/login.cy.ts

ts

supportFile: "cypress/support/e2e.ts"

This tells Cypress:

Before tests run, load common setup code from this file.

This is where we later add reusable commands like:

ts

cy.login()

ts

video: true, screenshotOnRunFailure: true

This means:

If a test fails, save screenshots and videos.

Very useful in Azure DevOps because you can see why the test failed.

ts

retries: { runMode: 1, openMode: 0 }

This means:

In pipeline mode, retry a failed test once.

In local interactive mode, don't retry.

This helps with small flaky issues in CI.

The command part

When you run:

bash

BASE_URL=example.com npm run cy:run

You are saying:

Run Cypress tests against example.com.

On Windows PowerShell:

powershell

\$env:BASE_URL="example.com"; npm run cy:run

Same idea, just Windows syntax.

In plain English

Step 2 sets up Cypress so that:

- **Tests live in cypress/e2e**
- **Test files use TypeScript**
- **Website URL comes from CI/CD**
- **Screenshots and videos are saved**
- **Failed tests retry once in pipeline runs**
- **Common reusable setup will load automatically**

Once this config is added, approve Step 2 and we'll move to Step 3: TypeScript Configuration.

tl-dr;

TL;DR:

We'll build a Cypress + TypeScript UI Automation Framework using:

- **Page Object Model**
- **Mochawesome reporting**
- **Azure DevOps pipeline**
- **CI/CD-driven environment configuration**

Framework flow:

1. Create project
2. Install Cypress + TypeScript
3. Configure Cypress
4. Add Page Object Model structure
5. Add reusable commands/utilities
6. Add sample UI tests
7. Add Mochawesome reports
8. Add Azure DevOps pipeline

Step 1 approval needed

Run these commands:

bash

```
mkdir cypress-typescript-ui-frameworkcd cypress-typescript-ui-frameworknpm init -ynpm install cypress typescript --save-devnpx cypress open
```

Then add this to package.json:

json

```
{ "scripts": {  "cy:open": "cypress open",  "cy:run": "cypress run"  }}
```

After this, approve Step 1 and we'll move to Step 2: Cypress Configuration.

approved step 1

Step 2: Cypress Configuration

Create or update cypress.config.ts in the project root.

ts

```
import { defineConfig } from "cypress";export default  
defineConfig({  e2e: {    baseUrl: process.env.BASE_URL,  
specPattern: "cypress/e2e/**/*.cy.ts",    supportFile:  
"cypress/support/e2e.ts",    setupNodeEvents(on, config) {      return  
config;    },  },  screenshotsFolder: "cypress/screenshots",  
videosFolder: "cypress/videos",  downloadsFolder:
```



```
"cypress/downloads", video: true, screenshotOnRunFailure: true,  
retries: { runMode: 1, openMode: 0, },  
defaultCommandTimeout: 10000, pageLoadTimeout: 60000,  
requestTimeout: 15000, responseTimeout: 30000, viewportWidth:  
1440, viewportHeight: 900,});
```

Why this setup?

- **baseUrl: process.env.BASE_URL supports Azure DevOps pipeline variables**
- **specPattern looks for test files under cypress/e2e**
- **supportFile loads reusable commands and global setup**
- **retries.runMode: 1 helps reduce minor CI flakiness**
- **Screenshots and videos are enabled for failure debugging**

Update package.json

Add these scripts:

json

```
{ "scripts": { "cy:open": "cypress open", "cy:run": "cypress run",  
"test:e2e": "cypress run", "test:e2e:chrome": "cypress run --  
browser chrome" }}
```

Validate Step 2

Run locally with a temporary BASE_URL.

macOS/Linux

bash

```
BASE_URL=[example.com](https://example.com) npm run cy:run
```

Windows PowerShell

powershell

```
$env:BASE_URL="[example.com](https://example.com)"; npm run  
cy:run
```

Expected result: Cypress starts successfully and looks for specs under:

text

cypress/e2e//*.cy.ts**

Approve Step 2 once this is done, and we'll proceed to Step 3: TypeScript Configuration.

ELI 15

Think of Step 2 as telling Cypress: "Where are my tests, what website should I test, and how should Cypress behave?"

Here's the simple version.

What cypress.config.ts does

This file is the main settings file for Cypress.

ts

baseUrl: process.env.BASE_URL

This means:

***Don't hardcode the website URL in the framework.
Get it from the pipeline or terminal instead.***

So locally you can run against:

text

example.com

But in Azure DevOps, the pipeline can provide:

text

qa.yourapp.com

That keeps the framework flexible.

Important settings

ts

specPattern: "cypress/e2e//*.cy.ts"**

This tells Cypress:

Look for test files inside cypress/e2e, and only run files ending with .cy.ts.

Example:

text

cypress/e2e/login/login.cy.ts

ts

supportFile: "cypress/support/e2e.ts"

This tells Cypress:

Before tests run, load common setup code from this file.

This is where we later add reusable commands like:

ts

cy.login()

ts

video: true, screenshotOnRunFailure: true

This means:

If a test fails, save screenshots and videos.

Very useful in Azure DevOps because you can see why the test failed.

ts

retries: { runMode: 1, openMode: 0 }

This means:

In pipeline mode, retry a failed test once.

In local interactive mode, don't retry.

This helps with small flaky issues in CI.

The command part

When you run:

bash

BASE_URL=example.com npm run cy:run

You are saying:

Run Cypress tests against example.com.

On Windows PowerShell:

powershell

\$env:BASE_URL="example.com"; npm run cy:run

Same idea, just Windows syntax.

In plain English

Step 2 sets up Cypress so that:

- **Tests live in cypress/e2e**
- **Test files use TypeScript**
- **Website URL comes from CI/CD**
- **Screenshots and videos are saved**
- **Failed tests retry once in pipeline runs**
- **Common reusable setup will load automatically**

Once this config is added, approve Step 2 and we'll move to Step 3: TypeScript Configuration.